SQL handbook

Barbara Argueta Vargas

What is SQL?

SQL (Structured Query Language) is the standard language for interacting with **relational databases.** It allows you to create, manipulate, and query data stored in database systems. It's essential for tasks like data analysis, software development, and business information management.

Why SQL?

- It's the global standard for working with data.
- It's easy to learn and use.
- It's widely used across industries like technology, finance, healthcare, education, and more.

What is a relational database?

A relational database organizes data into tables (rows and columns). Each table has a defined schema that determines its structure.

Basic concepts

Table components

- **Rows (Records):** Represent individual units of data.
- **Columns (Attributes):** Each column stores a specific type of data.
- **Primary Key:** A unique identifier for each row.
- **Foreign Key:** Establishes relationships between tables.

Common Data Types

- **Numeric:** INT, FLOAT, DECIMAL
- **Text:** CHAR, VARCHAR, TEXT
- **Dates and Times:** DATE, DATETIME, TIMESTAMP
- **Boolean:** BOOLEAN or BIT

Some good practices

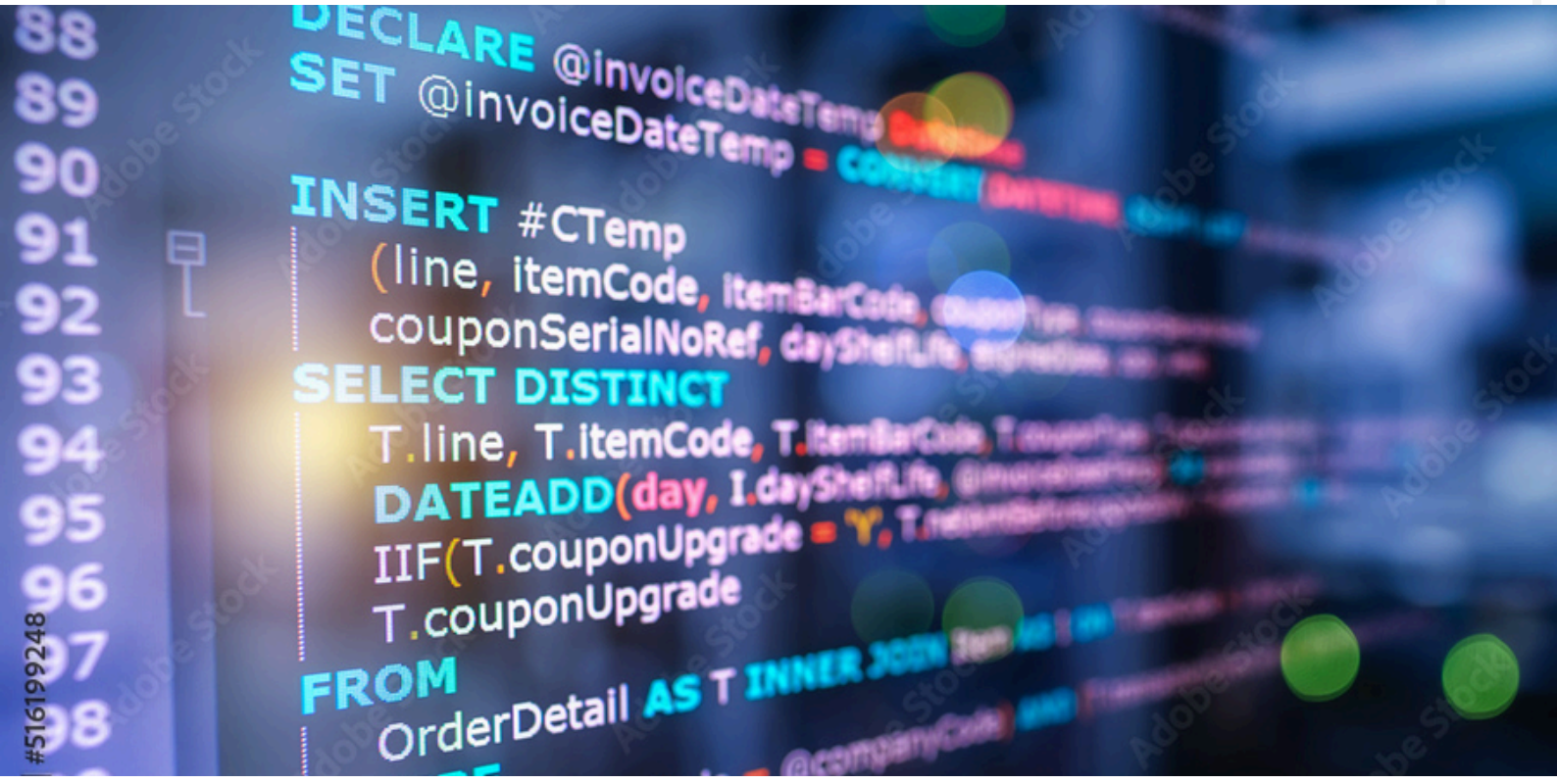
- **Indenting:** makes the SQL query visually structured and easier to follow.
- **SELECT*:** when using SELECT * , all columns will be returned in the same order as they are defined, so the returned data may change whenever the table definitions change.
- **Table Aliases:** it is more readable to use aliases instead of writing columns without table information.
- **SELECT DISTINCT:** is a handy way to remove duplicates from a query

SQL Commands

First things first...
types of SQL languages

1. **DDL (Data Definition Language):**
   Used to define and modify the structure of the database.
2. **DML (Data Manipulation Language):**
   Used to manipulate data within tables.
3. **DQL (Data Query Language):**
   Used to query data.

note: we're going to revise the three of them but the most used language is the DQL so this handbook will be focused on that specific language.



Essential commands

SELECT

The SELECT clause is used to specify the columns you want to retrieve from a table or result set. It defines what data will appear in the query output.

FROM

The FROM clause indicates the table(s) from which data is retrieved. It serves as the source of the data for the query.

WHERE

The WHERE clause filters rows based on a specified condition. Only rows that meet the condition(s) are included in the result set.

1. DDL

CREATE

```
CREATE TABLE table_name (
        column1 DATATYPE CONSTRAINT,
        column2 DATATYPE CONSTRAINT,
PRIMARY KEY (column1) );
```

MODIFY TABLES

```
ALTER TABLE table_name
ADD column_name DATATYPE;
```

DELETE

```
DROP TABLE table_name;
```

2. DML

INSERT

```
INSERT INTO table_name (column1, column2)
VALUES (value1, value2);
```

UPDATE

```
UPDATE table_name SET column1 = new_value
WHERE condition;
```

DELETE

```
DELETE FROM table_name
WHERE condition;
```

3. DQL

Now we know about basic commands such as SELECT, FROM and WHERE. We'll learn about logic operators and and aggregate functions.

AND

The AND operator combines two or more conditions in a WHERE clause, returning rows only if all the conditions are true. It is used to apply multiple filters to a query.

```
SELECT name, age
FROM employees
WHERE age > 30 AND department = 'Sales';
```

OR

The OR operator combines two or more conditions in a WHERE clause, returning rows if any of the conditions are true. It is used to broaden the query results by allowing multiple criteria to pass.

```
SELECT name, age
FROM employees
WHERE age > 50 OR department = 'HR';
```

EXAMPLE OF COMBINATION

```
SELECT name, age
FROM employees
WHERE (age > 50 OR department = 'HR') AND salary
> 50000;
```

OPERATORS FOR NUMERICAL DATA

Operator	Condition
=, !=, <, <=, >, >=	Standard numerical operators
BETWEEN ... AND ...	Number is within range of two values (inclusive)
NOT BETWEEN ... AND ...	Number is nor within range of two values (inclusive)
IN (...)	Number exists in a list
NOT IN (...)	Number does not exist in a list

OPERATORS FOR TEXT DATA

Operator	Condition
=	Case sensitive exact string comparison
!= or <>	Case sentitive exact string inequality comparison
LIKE	Case sensitive exact string comparison
NOT LIKE	Case insensitive exact string inequality comparison
%	Used anywhere in a string to match a sequence of zero or more characters (only with LIKE or NOT LIKE)
_	Used anywhere in a string to match a single character (only with LIKE or NOT LIKE)
IN (...)	String exists in a list
NOT IN (...)	String does not exist in a list

FILTERING AND SORTING

The DISTINCT keyword is used in the SELECT statement to remove duplicate rows from the result set, ensuring that only unique values are returned.

SELECT DISTINCT department FROM employees;

AGGREGATE FUNCTIONS

- COUNT: Counts rows.
- SUM: Sums values.
- AVG: Calculates the average.
- MAX: Finds the maximum value.
- MIN: Finds the minimum value.

GROUP BY

The GROUP BY clause groups rows with the same values in specified columns into summary rows. It is often used with aggregate functions (like COUNT, SUM, AVG, MAX, or MIN) to perform calculations on groups of data.

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

ORDER BY

The ORDER BY clause sorts the rows in the result set based on one or more columns, either in ascending (ASC) or descending (DESC) order.

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

LIMIT

The LIMIT clause is used to specify the maximum number of rows to return in a query result.

OFFSET

The OFFSET clause specifies the number of rows to skip before starting to return rows from the query result. It is typically used with LIMIT for implementing pagination.

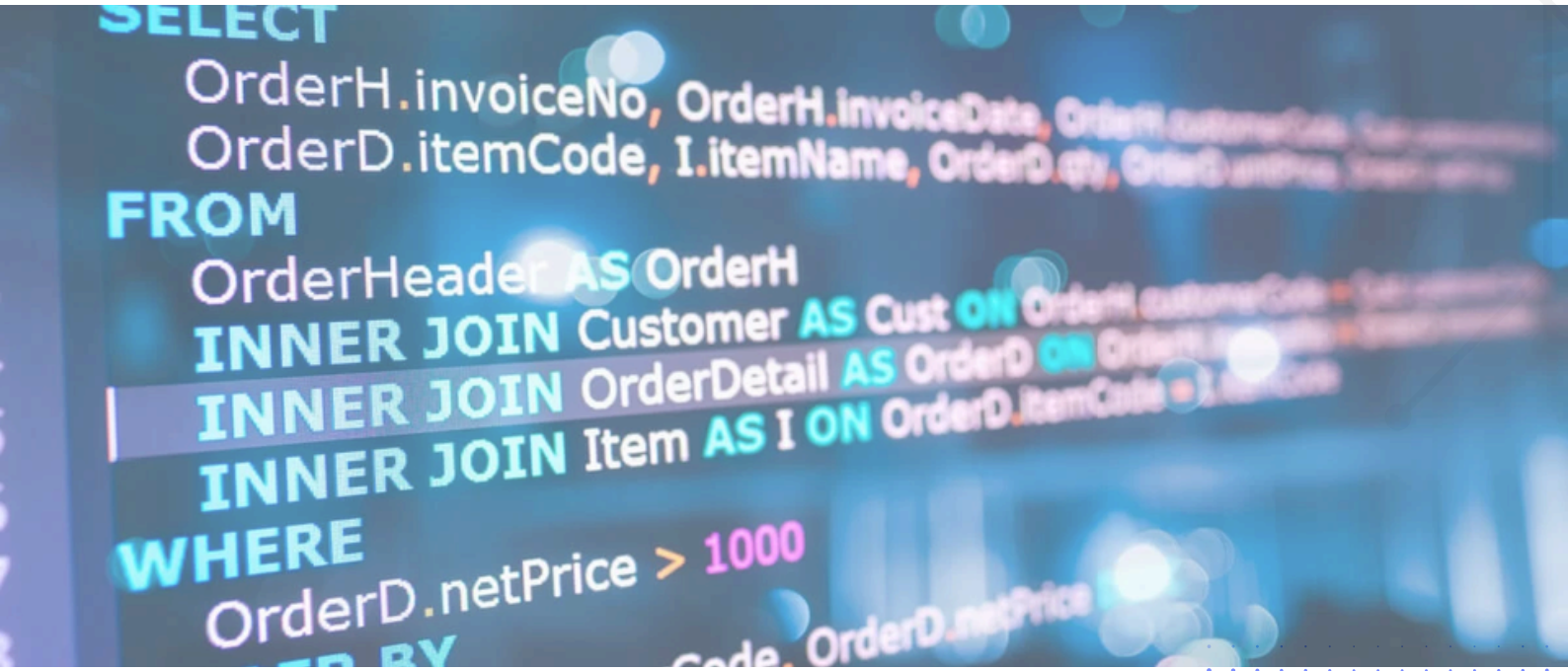
Relationships between tables

What is a SQL JOIN?

A SQL JOIN clause is used to query and access data from multiple tables by establishing logical relationships between them. It can access data from multiple tables simultaneously using common key values shared across different tables.

Types of JOINS

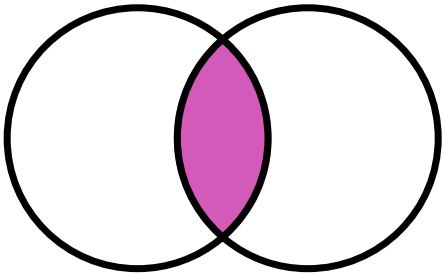
- CROSS JOIN
- **INNER JOIN**
- **LEFT OUTER JOIN**
- **RIGHT OUTER JOIN**
- **FULL OUTER JOIN**
- SELF JOIN



INNER JOIN

The INNER JOIN keyword selects all rows from both the tables as long as the condition is satisfied. This keyword will create the result-set by combining all rows from both the tables where the condition satisfies i.e value of the common field will be the same.

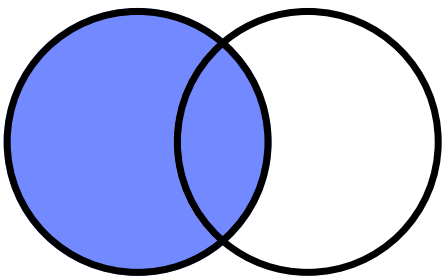
```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
INNER JOIN table2
ON table1.matching_column = table2.matching_column;
```



LEFT JOIN

The LEFT JOIN returns all the rows of the table on the left side of the join and matches rows for the table on the right side of the join. For the rows for which there is no matching row on the right side, the result-set will contain null.

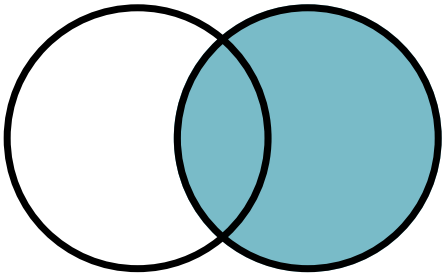
```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
LEFT JOIN table2
ON table1.matching_column = table2.matching_column;
```



RIGHT JOIN

The RIGHT JOIN returns all the rows of the table on the right side of the join and matching rows for the table on the left side of the join. It is very similar to LEFT JOIN for the rows for which there is no matching row on the left side, the result-set will contain null.

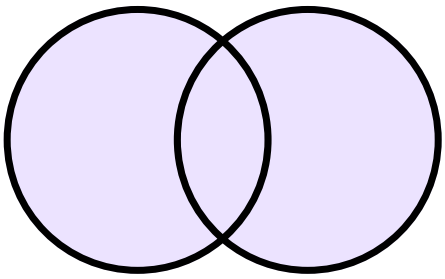
```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```



FULL JOIN

The FULL JOIN creates the result-set by combining results of both LEFT JOIN and RIGHT JOIN. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain NULL values.

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
FULL JOIN table2
ON table1.matching_column = table2.matching_column;
```



JOIN EXAMPLE

Here's an example using various joins and clauses that we just learned:

With the following tables extract the employee name, their department and manager name.

employee_id	name	department_id	manager_id
1	Alice	10	101
2	Bob	20	102
3	Charlie	10	NULL

department_id	department_name
10	Sales
20	IT
30	HR

manager_id	manager_name
101	Sarah
102	John
103	Emma

QUERY

```
SELECT
    e.name AS employee_name,
    d.department_name,
    m.manager_name
FROM
    Employees e
INNER JOIN Departments d
    ON e.department_id = d.department_id
LEFT JOIN Managers m
    ON e.manager_id = m.manager_id
FULL OUTER JOIN Departments d2
    ON d.department_id = d2.department_id
WHERE e.department_id IS NOT NULL;
```

RESULT

employee_name	department_name	manager_name
Alice	Sales	Sarah
Bob	IT	John
Charlie	Sales	NULL
NULL	HR	NULL

THANK YOU :)

Barbara Argueta Vargas

References:
https://sqlbolt.com/lesson/introduction
https://www.geeksforgeeks.org/sql-join-set-1-inner-left-right-and-full-joins/
https://mteheran.dev/buenas-practicas-en-sql/
https://www.javatpoint.com/types-of-sql-join